📘 BOOTCAMP SURVIVAL GUIDE

Data Analyst (DA) • Data Scientist (DS) • Data Engineer (DE)

> Ditulis dengan mindset: **anak bootcamp → siap kerja → ngerti konsep + bisa implement basic → tahu arah next level**
> Fokus: **practical + intuition**, bukan teori berat

🟦 SECTION 1 — DATA ANALYST (DA)

1. Python for Data Analysis (Essentials Only)

Basic Syntax (secukupnya)

```
print("Hello World")

def add(a, b):
    return a + b
```

Pandas (Core Weapon)

```
import pandas as pd

df = pd.read_csv("data.csv")

df.head()
df.info()
df.describe()

df['column']
df[['col1', 'col2']]
```

```
df[df['age'] > 30]

df.groupby('category')['sales'].sum()

df.isnull().sum()
df.dropna()
df.fillna(0)
```

Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(df['age'])
sns.boxplot(x='category', y='sales', data=df)

plt.show()
```

👉 Fokus:

- Pandas = manipulasi data
- Seaborn = quick insight
- Matplotlib = control

---

2. SQL (Cheatsheet Level DA → Advanced Enough)

Basic

```
SELECT name, age
FROM users
WHERE age > 25
ORDER BY age DESC;
```

Aggregation

```
SELECT category, COUNT(*), AVG(price)
FROM products
```

```
GROUP BY category;
```

JOIN

```
SELECT o.id, c.name
FROM orders o
JOIN customers c ON o.customer_id = c.id;
```

Subquery

```
SELECT name
FROM users
WHERE salary > (SELECT AVG(salary) FROM users);
```

CTE (WITH)

```
WITH sales_cte AS (
    SELECT customer_id, SUM(amount) AS total
    FROM orders
    GROUP BY customer_id
)
SELECT *
FROM sales_cte
WHERE total > 1000;
```

Window Function 🔥

```
SELECT
    name,
    salary,
    RANK() OVER (ORDER BY salary DESC) as rank
FROM employees;
```

👉 WAJIB PAHAM:

- GROUP BY vs WINDOW FUNCTION
- JOIN logic

- CTE = readable query

---

3. Web Scraping (Basic)

BeautifulSoup

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
res = requests.get(url)

soup = BeautifulSoup(res.text, 'html.parser')

titles = soup.find_all('h2')
for t in titles:
    print(t.text)
```

Selenium (dynamic web)

```
from selenium import webdriver

driver = webdriver.Chrome()
driver.get("https://example.com")

element = driver.find_element("tag name", "h1")
print(element.text)
```

---

4. Business Knowledge (Basic DA Mindset)

👉 Lu bukan cuma ngolah data, tapi jawab:

- "So what?"
- "Impact ke bisnis apa?"

Contoh:

- Revenue turun → karena apa?

- Customer churn → kenapa?
- Campaign gagal → faktor apa?

👉 Framework:

- Revenue = Price × Quantity
- Funnel:
    - Awareness → Conversion → Retention

---

5. Statistics (Basic for DA)

Descriptive

- Mean, Median, Mode
- Std Dev
- Distribution

Inferential

- Hypothesis Testing
- T-test
- P-value

👉 Intuisi:

- "Apakah perbedaan ini real atau cuma kebetulan?"

---

6. Tableau (Basic)

👉 Fokus:

- Bar chart
- Line chart
- Filter
- Dashboard

👉 Prinsip:

- 1 chart = 1 insight
- Jangan over-design
- Story > visual cantik

🟥 SECTION 2 — DATA SCIENTIST (DEEP DIVE VERSION - RAG READY)

> Goal section ini:
> **Dalam beberapa jam belajar → student ngerti "cara kerja model", bukan cuma pakai library**

---

1. MACHINE LEARNING FUNDAMENTAL

Problem Types

Type	Output	Contoh
Regression	Continuous	Harga rumah
Classification	Category	Churn (yes/no)
Clustering	Group	Segmentasi customer

General ML Workflow (REAL VERSION)

1. Define problem (business → ML problem)
2. Data cleaning (missing, outlier)
3. EDA (pattern, anomaly)
4. Feature engineering (**MOST IMPORTANT**)
5. Model selection
6. Training
7. Evaluation
8. Iteration

---

2. FEATURE ENGINEERING (LEVEL UP)

2.1 Encoding

One Hot Encoding

```
pd.get_dummies(df['category'])
```

➡️ Problem:

- Dimensi bisa meledak

Label Encoding

```
from sklearn.preprocessing import LabelEncoder
```

➡️ Hati-hati:

- Model bisa anggap ada urutan (padahal tidak)

2.2 Scaling

Standardization

```
(x - mean) / std
```

➡️ Wajib untuk:

- KNN
- SVM
- Logistic Regression

2.3 Feature Creation

Contoh:

- price_per_unit = total_price / quantity
- days_since_last_purchase

➡️ Ini sering lebih impactful dari model

2.4 Handling Missing Values

- Drop

- Mean/Median
- Model-based imputation

2.5 Outlier Handling

- IQR method
- Z-score

---

3. REGRESSION (MATHEMATICAL INTUITION)

Linear Regression

Formula:

$$y = \beta 0 + \beta 1x1 + \beta 2x2 + ... + \varepsilon$$

Objective:

Minimize error:

$$MSE = (1/n)\ \Sigma(y\_true - y\_pred)^2$$

Assumptions:

- Linear relationship
- No multicollinearity
- Homoscedasticity

Regularization

Ridge (L2):

$$Loss = MSE + \lambda \Sigma w^2$$

Lasso (L1):

$$Loss = MSE + \lambda \Sigma |w|$$

➡️ Lasso bisa feature selection

---

4. CLASSIFICATION (CORE CONCEPT)

4.1 Logistic Regression 🔥

➡️ Bukan regression!

Function:

```
P(y=1) = 1 / (1 + e^(-z))
```

➡️ Output = probability

4.2 KNN

➡️ Cara kerja:

- Ambil K tetangga terdekat
- Majority voting

➡️ Weakness:

- Lambat di data besar
- Sensitif ke scaling

4.3 Naive Bayes

Formula:

```
P(A|B) = P(B|A)P(A) / P(B)
```

➡️ Asumsi:

- Feature independen (jarang true, tapi tetap works)

4.4 SVM

➡️ Tujuan:

- Cari hyperplane terbaik

➡️ Konsep penting:

- Margin
- Support vector

➡️ Kernel trick:

- Linear
- RBF

4.5 Decision Tree

➡️ Cara kerja:

- Split berdasarkan feature

Metric:

- Gini
- Entropy

➡️ Problem:

- Overfitting

5. ENSEMBLE LEARNING 🔥

5.1 Bagging

Random Forest:

- Banyak tree
- Random subset data

➡️ Reduce variance

5.2 Boosting

Gradient Boosting:

- Model belajar dari error sebelumnya

➡️ Reduce bias

---

6. MODEL EVALUATION (WAJIB KUAT)

Regression

- MAE
- MSE
- RMSE

Classification

Confusion Matrix:

- TP, FP, TN, FN

Metrics:

- Accuracy
- Precision
- Recall
- F1-score

➡️ Insight:

- Precision tinggi → sedikit false positive
- Recall tinggi → sedikit false negative

7. PIPELINE & TUNING

Pipeline

```
Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])
```

Hyperparameter Tuning

- GridSearch
- RandomSearch

Cross Validation

➡️ Hindari overfitting

Data Balancing ⚠️

Techniques:

- SMOTE
- Undersampling

Trade-off:

- Data jadi tidak natural
- Bisa misleading

---

8. DIMENSIONALITY REDUCTION

PCA (Principal Component Analysis)

➡️ Intuisi:

- Cari arah variance terbesar

➡️ Output:

- Feature baru (orthogonal)

➡️ Kegunaan:

- Reduce dimension
- Visualisasi

---

9. UNSUPERVISED LEARNING

K-Means

Steps:

1. Pilih K
2. Assign cluster
3. Update centroid

➡️ Problem:

- Harus tentukan K

Elbow Method:

➡️ Cari K optimal

---

10. TIME SERIES

ARIMA

- AR (AutoRegressive)
- I (Integrated)
- MA (Moving Average)

SARIMA

➡️ Tambah seasonality

Prophet (Meta)

➡️ Lebih user-friendly
➡️ Cocok untuk bisnis

---

11. MLOPS (BOOTCAMP LEVEL)

Basic Concept:

- Training → Save model → Deploy → Monitor

Tools:

- FastAPI
- Docker

---

12. AI ETHICS

⚠️ Penting:

- Bias model
- Data leakage
- Fairness

---

13. ANN (NEURAL NETWORK)

Struktur:

- Input layer
- Hidden layer
- Output layer

Activation Function:

- ReLU
- Sigmoid
- Softmax

Training:

- Forward propagation
- Loss calculation
- Backpropagation

---

14. CNN (COMPUTER VISION)

Komponen:

- Convolution layer
- Pooling
- Fully connected

➡️ Gunanya:

- Deteksi pola visual

---

15. NLP (DEEP FOCUS)

Classic NLP

TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

RNN / LSTM 🔥

➡️ Cocok untuk sequence

Intuisi:

- Memory dari kata sebelumnya

Problem:

- Vanishing gradient

LSTM solution:

- Gate mechanism

🎯 FINAL TAKEAWAY (DS CORE)

Kalau cuma inget 5 hal:

Feature Engineering > Model
Evaluation metric itu krusial
Overfitting adalah musuh utama
Model itu alat, bukan tujuan
Selalu balik ke business problem

🟩 SECTION 3 — DATA ENGINEER (BOOTCAMP → HANDS-ON READY)

> Goal section ini:
> **Lu bukan cuma tau konsep, tapi bisa kebayang "gimana bikin pipeline end-to-end"**

---

1. BIG PICTURE DATA ENGINEERING

Flow Dunia Nyata

```
Source → Ingestion → Storage → Processing → Serving → Visualization
```

Contoh:

- API → Kafka → Data Lake → Spark → Warehouse → Dashboard

Batch vs Streaming

Type	Contoh	Tools
Batch	Daily report	Airflow
Streaming	Real-time click	Kafka

---

2. DOCKER (WAJIB BANGET)

Kenapa penting?

- "Works on my machine" problem → hilang
- Semua environment konsisten

Basic Command

```
docker build -t my-app .
docker run -p 8000:8000 my-app
```

```
docker ps
docker stop <container_id>
```

Dockerfile Example

```
FROM python:3.10

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

CMD ["python", "app.py"]
```

Docker Compose (Multi Service)

```
version: '3'
services:
  app:
    build: .
    ports:
      - "8000:8000"
  db:
    image: postgres
```

👉 Insight:

- DE hampir selalu pakai multi-service (DB + app + scheduler)

---

3. APACHE AIRFLOW (CORE PIPELINE)

Concept

- DAG = workflow
- Task = step dalam workflow

Struktur DAG

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

def extract():
    print("Extract data")

def transform():
    print("Transform data")

def load():
    print("Load data")

with DAG("etl_pipeline", start_date=datetime(2024,1,1), schedule="@daily"

    t1 = PythonOperator(task_id="extract", python_callable=extract)
    t2 = PythonOperator(task_id="transform", python_callable=transform)
    t3 = PythonOperator(task_id="load", python_callable=load)

    t1 >> t2 >> t3
```

Best Practice

- Idempotent (run ulang ga error)
- Logging jelas
- Modular task

Use Case Nyata

- Ambil data API tiap hari
- Clean data
- Masuk ke database

---

4. APACHE KAFKA (STREAMING)

Concept Core

- Producer → kirim data
- Broker → tempat data
- Consumer → ambil data

Analogi

- Kafka = "WhatsApp untuk data"
- Producer = sender
- Consumer = reader

Use Case

- Clickstream
- Real-time analytics

Python Example

```
from kafka import KafkaProducer

producer = KafkaProducer(bootstrap_servers='localhost:9092')
producer.send('topic_name', b'hello world')
```

Insight

- Kafka itu powerful tapi kompleks
- Bootcamp level → cukup ngerti flow

---

5. NOSQL DATABASE

Kenapa NoSQL?

- Schema fleksibel
- Cocok untuk data tidak terstruktur

Tipe:

- Document (MongoDB)
- Key-value (Redis)
- Search (Elasticsearch)

---

6. MONGODB (DOCUMENT DB)

Basic Query

```
db.users.insert_one({"name": "John", "age": 25})

db.users.find({"age": {"$gt": 20}})
```

Struktur Data

```
{
  "name": "John",
  "orders": [
    {"item": "A", "price": 10},
    {"item": "B", "price": 20}
  ]
}
```

Insight

- Nested data = powerful
- Tapi bisa messy kalau ga design

---

7. ELASTICSEARCH + KIBANA

Elasticsearch

- Full-text search engine
- Query cepat

Contoh Query

```
GET /products/_search
{
  "query": {
    "match": {
      "name": "laptop"
    }
  }
}
```

Kibana

- Dashboard untuk Elasticsearch

Use Case

- Log monitoring
- Search engine

---

8. PYSPARK (BIG DATA PROCESSING)

Kenapa Spark?

- Data terlalu besar untuk Pandas

Basic Example

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("test").getOrCreate()

df = spark.read.csv("data.csv", header=True)

df.show()
df.groupBy("category").count().show()
```

Concept

- Distributed computing
- Lazy evaluation

Insight

- Jangan pakai kalau data kecil
- Overkill

---

9. DBT (DATA TRANSFORMATION)

Tujuan

- Transform data di warehouse

Contoh Model

```
SELECT *
FROM raw_orders
WHERE status = 'completed'
```

Kenapa dbt?

- SQL jadi modular
- Version control

---

10. SNOWFLAKE (DATA WAREHOUSE)

Concept

- Cloud warehouse
- Scalable

Basic Query

```
SELECT * FROM sales;
```

Insight

- Storage & compute terpisah
- Pay as you go

---

11. AWS GLUE (VERY BASIC)

Apa itu?

- ETL service di AWS

Flow

- Extract → Transform → Load

Use Case

- Integrasi data AWS ecosystem

---

12. HADOOP (VERY BASIC)

Concept

- Distributed storage (HDFS)
- MapReduce

Insight

- Predecessor Spark
- Sekarang lebih jarang dipakai langsung

---

13. END-TO-END PIPELINE (WAJIB NGERTI)

Contoh Real Pipeline

1. Scrape data (Python)
2. Kirim ke Kafka
3. Simpan ke MongoDB
4. Process pakai Spark
5. Simpan ke Snowflake
6. Visualisasi di Kibana/Tableau

Simplified Bootcamp Version

1. API → Pandas
2. Clean data
3. Save ke PostgreSQL
4. Schedule pakai Airflow

---

14. COMMON MISTAKES

❌ Langsung pakai tool berat
❌ Ga ngerti flow data
❌ Overengineering

---

15. FINAL TAKEAWAY (DE CORE)

Kalau cuma inget ini:

1. Data flow > tools
2. Airflow = batch king
3. Kafka = streaming king
4. Docker = wajib
5. Simplicity > complexity

🎯 END GOAL DE

Bukan jadi infra engineer hardcore.

Tapi:

- Ngerti pipeline
- Bisa connect system
- Bisa support DS/DA

🔥 FINAL NOTE

Kalau bingung prioritasi:

1. DA → SQL + Pandas
2. DS → Feature Engineering + Modeling
3. DE → Airflow + Basic Pipeline

👉 Real world:

- Skill = kombinasi
- Bukan silo

🎯 END GOAL

Bukan jadi jago teori.

Tapi:

- Bisa solve problem
- Bisa explain hasil
- Bisa deploy (basic)